

Behavioral Synthesis of ICs

Design Description

Credentials: majority of information are from the following sources:

1. Khalid, Ayesha, Goutam Paul, and Anupam Chattopadhyay. *Domain Specific High-Level Synthesis for Cryptographic Workloads*. Springer Singapore, 2019.
2. Zwolinski, Mark. *Digital system design with SystemVerilog*. Pearson Education, 2009.

Outline

1. Generation Through Automation
2. Hardware Description Languages
3. VHDL Through Examples

Objectives

- Understand the concept of generation through automation in IC design
- Explore the key aspects of hardware description languages (HDLs)
- Gain practical knowledge of VHDL through examples

Generation Through Automation

Generation Through Automation

- To overcome the **complexity** of modern electronic system design, **automation** is the solution.
- The system is modeled at a **high abstraction level** as an executable specification of the design for early design space exploration.
- This abstract modeling is required for **High Level Synthesis** (HLS) or behavioral synthesis techniques to bridge the gap between algorithm developers and hardware designers by supplying a platform for co-development.
- Consequently, design exploration is accelerated, and **time-to-market** is shortened.
- The HLS techniques are carried out either through **High Level Languages** (HLLs) or through **Architecture Description Languages** (ADLs).

Architecture description: Architecture Description Languages

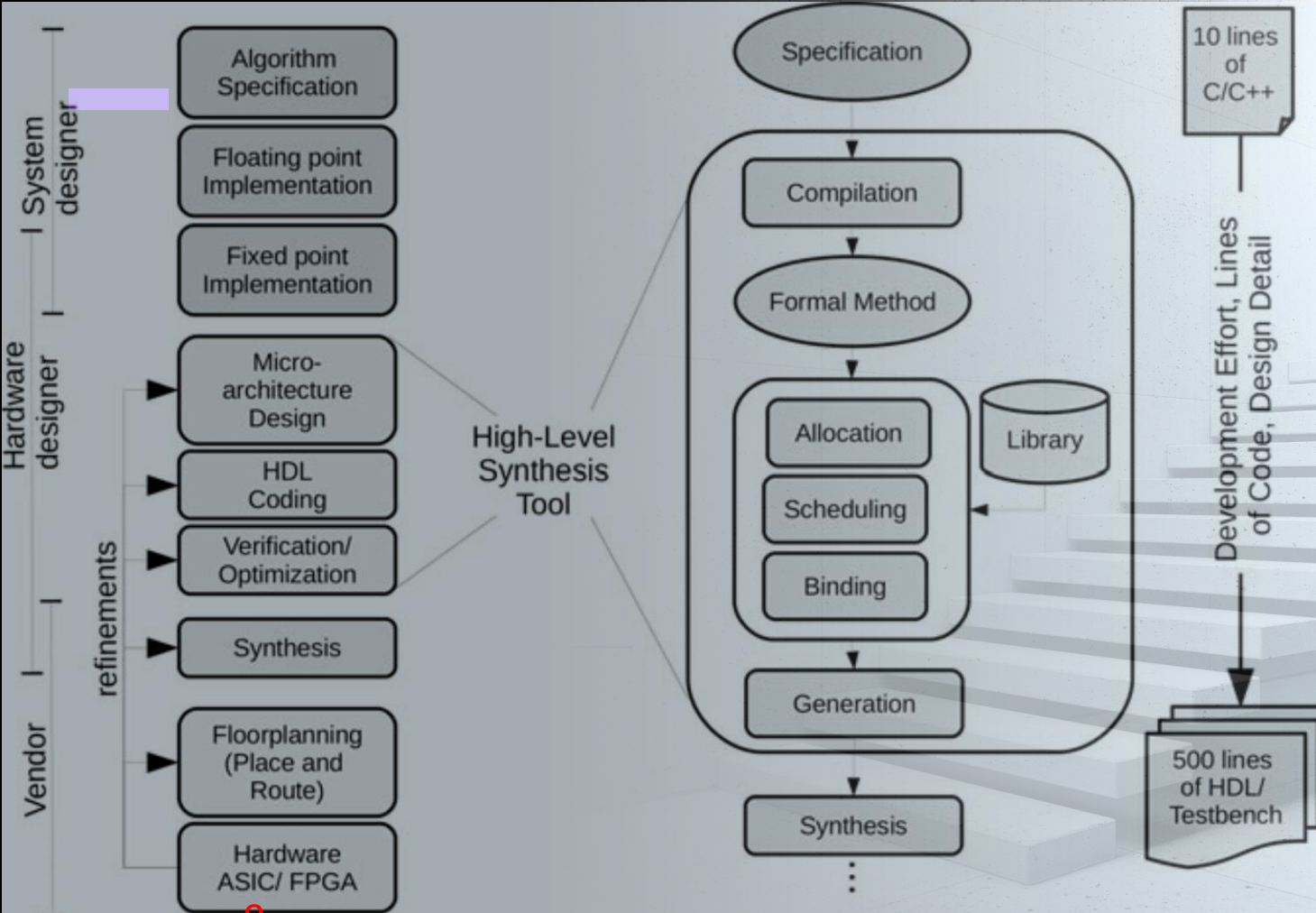
- In contrast, HLS tools that take ADLs (**Architecture Description Languages**) description of the design let the designer choose an architecture choice over the entire design space.
- The tool-flow is suited for **Application Specific Instruction set Processors** (ASIPs) as the designer describes the architecture in a higher abstraction by ADLs, as well as the structures for **Instruction Set Architecture** (ISA), latency, op-codes, pipeline, memory architecture etc.
- The **quality** and **ease of exploration** directly depends on the **language flexibility** of ADL.
- Ideally, the ADL development environment should come with a suite of aiding tools to enable high level simulations where critical design decisions are guided early in the design by simulation loops.
- Some high-level processor modeling languages include
 - LISA ADL (Synopsys Processor Designer),
 - nML,
 - EXPRESSION.

Algorithm description: High Level Language (HLL)

- The architecture engineers can, however, relate better to the ADLs.
- For **application specific workloads**, e.g., cryptographic workloads, the first approach or HLS is more suited.
- HLS through a HLL requires **sequential specification** of the system by the user in a **higher abstraction level language**.
- The HLS tool converts this **untimed design** into a **fully timed digital hardware specifications**.
- The tool may be directed to architect the design considering various **constraints**, e.g., latency, resources, target platform, power budget, etc.
- The tool then explores various design possibilities obeying these constraints, choosing the level of **parallelism** against all **iterative structures** and **data dependencies**.
- Various **trade-offs** and **optimizations** are evaluated to finally reach a **synthesizable**, efficient hardware description and its respective verification environment.

Algorithm description: High Level Language (HLL)

- Conformity to **constraints** might not always be possible though.
- Whenever possible, the tool develops high quality hardware designs exploiting the design flexibility and consequently application specific hardware mapped to ASIC/FPGA is naturally suitable to be undertaken by this design methodology.
- The user has limited (or indirect control through constraints specifications) over the architectural choices of generated architecture, hence the design space exploration to be done by the tool remains limited.
- Noticeable examples from industry and academia include
 - Synopsys Symphony C,
 - GAUT,
 - Xilinx Vitis HLS,
 - Mentor Graphics Catapult C



Steps of High Level Synthesis

Figure 1. The traditional VLSI development flow (left) against the High Level Synthesis (HLS) based development flow (right)

Steps of High-Level Synthesis

Basic steps of high-level synthesis are as follow:

- **Compilation into a Formal Model:** The high-level specification is first **parsed** and converted to an **Intermediate Representation** (IR).
 - IR is generally a **Data Flow Graph** (DFG) representing the **data dependencies** between operations.
 - Figure 1 shows a simplistic example of the conversion of C specifications into a DFG.
 - An extended DFG, handling the control dependencies in addition to data dependencies called **Control Data Flow Graph** (CDFG) is also being used as IR.

Steps of High-Level Synthesis

- **Allocation:** For each operation in the IR, a **hardware resource** is allocated (e.g., function unit, storage element, connectivity buses) from the RTL component **library** as shown in Figure 1 (right).
 - Since the library components have other characteristics (**area**, **delay**, **power**), the selection is based on the user specified design **constraints**.
- **Scheduling:** Scheduling creates a clock scheme assigning time cycles or control steps on operations based on time/resource constraints.
 - Depending upon the functional unit on which the operation is mapped, an operation may be extended to **multiple cycles** or **multiple operations** may be merged in on **clock step**.
 - The simplest unconstrained scheduling algorithms are **ASAP** (**As Soon As Possible**) and **ALAP** (**As Late As Possible**).

Steps of High-Level Synthesis

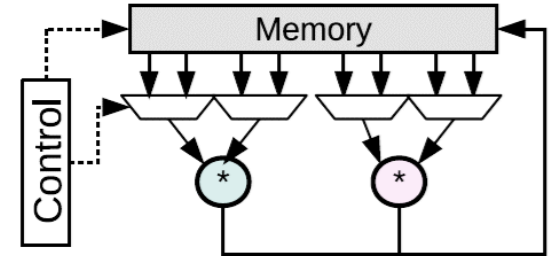
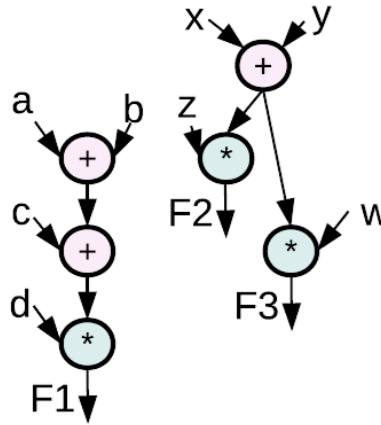
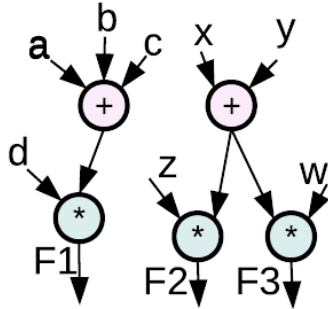
- **ASAP schedules** an operation for execution on a control step as soon as all its dependencies are resolved and hence exhibits a minimum latency unconstrained scheduling.
- **ALAP scheduling** schedules the operation for the last control step back to the first. An operation is delayed for execution till it is required for a next operation.
- More realistic scheduling constraints are **minimum latency under constrained resource (ML-RC)** and **minimum resource under latency constraints (MR-LC)**.
- These scheduling problems are **NP-hard**, consequently several Iterative **Heuristic Methods** are employed to solve them.
- Some most popularly used algorithms are **List Scheduling (LS)**, **Force-Directed Scheduling (FDS)** and **Simulated Annealing**.
- Figure 2 scheduling of the formal model into a clock scheme with resources constrained to just one Adder and one Multiplier.

Steps of High-Level Synthesis

→ Step of HLS:

- a. High-level language description
- b. Compilation into a DFG
- c. Minimum latency under constrained resource scheduling (1 Adder, 1 Multiplier) scheduling
- d. HDL architecture generation

$F1 = (a+b+c)*d$
 $F2 = (x+y)*z$
 $F3 = (x+y)*w$



a) High level language description

b) Copilation into Data Flow Graph (DFG)

c) Constrained resource scheduling

d) Hardware Architecture

Steps of High-Level Synthesis

- **Binding**: Binding completes the hardware architecture.
- This step binds all **operations** to **functional units** with an optimized selection, whenever possible.
- **Variables** that maintain values across **multiple cycles** are bound to respective **storage elements** while variables having non overlapping or mutually exclusive lifetimes are bound to the same storage elements. This ensures resource reuse.
- All **transfers** are binded to **buses**.
- Figure 2.d shows the complete architecture after binding with storage elements, functional units and buses.

Steps of High-Level Synthesis

- **Generation:** The HDL generation of the architecture model is the last step of HLS tool-flow.
- The generated HDL is generally partitioned into a **controller** and **datapath** and is implemented by **register transfer language (RTL)** components.
- The **datapath** comprises of all the **storage elements**, a set of **functional units**, and **interconnect elements**.
- The datapath can be **pipelines** and **consequently** may have input and output registers.
- The **controller** is a **Finite State Machine (FSM)** orchestrating datapath flow by setting the control signals for the functional units, registers, multiplexers etc. within the datapath.
- The datapath may return **status signals** back to the controller as required.

HLS: A Brief Retrospection

- The quest to enable hardware design modeling at a higher abstraction level, along with its optimization and implementation has been undertaken by several generations of CAD researchers, both from industry and academia.
- Some of the earliest research efforts were launched as early as 1970s while the industry was still adapting to the CAD systems; this era is termed as the generation 0 of HLS.
- A pioneering HLS tool was developed at Carnegie Mellon university (called [CMU Design automation or CMU-DA](#)), which accepted the behavioral description as ISPL (**Instruction Set Processor Language**).
- This description is transformed into an IR data-flow called the Value Trace before developing HDL level implementation

Generation 1 of HLS

- During the first generation of HLS (1980s-early 1990s) basic concepts of HLS were explored and some groundbreaking research work was published.
- One of the earliest HLS tools commercially available was [CATHEDRAL-II Silicon Compiler](#) for Digital Signal Processing (DSP) algorithms.
 - It accepted behavioral specifications in a domain specific language, namely SILAGE, that was specifically designed for the synthesis of DSP hardware with support for customized data types and easy transformations into different data types.
 - It used a programmable, bit parallel **VLIW architecture** for application mapping.
 - Several procedural optimizations were performed in the **Scheduling of microcode**.
 - The tool serves as an application specific silicon compiler that exploited well the defined target architecture design and led to efficient layout synthesis from a system point of view.

Generation 1 of HLS

- The first generation of HLS failed to be a commercial success due to several reasons.
- The idea of **HLS was hard to undertake** at that time by the designers since RTL synthesis adaptation was then just beginning.
- Moreover, learning a **new input language** with an unfamiliar design approach was not appreciated.
- Most of the tools of that generation relied on **simplistic allocation and scheduling algorithms** and consequently produced a lower QoR (Quality of Results).
- Moreover, the domain specialization focused on the DSP applications, that was not suitable for the vast majority of ASIC designers.

Generation 2 of HLS

- The second generation of HLS (mid 1990s-early 2000s) is marked by a wide adoption of **RTL-based design flows** and a simultaneous good deal of interest from the major **EDA companies towards HLS**.
- In 1995, **Synopsys** announced Behavioral Compiler that generated RTL implementations from behavioral hardware description language (HDL) code and was seamlessly connected to downstream tools.
- The compiler regularly used allocation and binding optimizations on the arithmetic and other functional units in conventional logic optimization.
- **Cadence** launched an industrial tool, the **Visual Architect Tool**, for the system level design of a system, described in behavioral VHDL for realization in FPGAs.
- The tool was enhanced with **precise timing handling** and also exploited feedback of **technology information** for better high-level synthesis.
- A similar tool was introduced by **Mentor Graphics** for HLS called **Monet**.

Generation 2 of HLS

- These tools received wide attention but failed to replace the traditional RTL design methodology due to following reasons:
 - The HLS required learning behavioral HDL language as input, consequently, the popularity of these tools remained confined to hardware designers.
 - The results of these tools were **hard to validate** due to absence of formal methods at that time.
 - Since HLS is more suited to dataflow and signal processing oriented flows, the results remained **suboptimal for control oriented dataflows**.
- Consequently, the generation 2 tools failed to expand market of HLS to algorithms and software developers.

The Current Generation of HLS

- A new generation of HLS tools emerged from 2000 with proposals both from academia and industry.
- This generation was able to overcome most of the shortcomings that their predecessors had and consequently had better industrial adaptation.
- To start with, most of these tools relied on C, C++ or C-like languages (e.g., MATLAB) to acquire the design specifications.
- This feature made them more accessible to system designers as it enabled **software/hardware co-design** and **co-verification**.
- Additionally, most of these tools **focused on dataflow** and **DSP domain applications** which are well suited for HLS tool flows, resulting in an improved QoR.
- Another noticeable feature of these tools is that many of them are **exclusive to a platform** (e.g., FPGAs, ASICs, DSPs). Consequently the target architecture is efficiency exploited at all design levels.

The Current Generation of HLS: Mentor Graphics

- [Mentor Graphics Catapult C Synthesis](#): Coming from a major EDA vendor, Catapult C was designed and enhanced in an entire era of research and development.
- It takes untimed **ANSI C/C++** descriptions as input to generate RTL (VHDL, Verilog) targeted for both ASICs and FPGAs.
- It accepts timing, target technology and area as **constraints**. Other constraints include interface synthesis directives to indicate dataflows into or out of the hardware design, loop directives for loop unrolling and adding parallelism to the design, memory directives for storage constraints and memory architecture and resource constraint to limit hardware resources.
- The tool carries out a **Hierarchical Synthesis** to simplify the design integration, hierarchy also helps to exploit function level (or task-level) parallelism.
- A technology-driven **scheduling** and **allocation** is carried out to generate pipelined, multi-block subsystems in **synthesizable RTL** along with **systemC transaction-level models (TLM)** and **wrappers** for simulation and verification.
- Application suitability, both for **control oriented** and **data oriented** workflows is claimed, along with wide endorsement by leading semiconductor vendors.
- In 2011 [Catapult C](#) was acquired by Calypto Design Systems, and later (2015) Calypto Design Systems was acquired by Mentor Graphics (**Siemens**).

The Current Generation of HLS: Xilinx

- [Xilinx High Level Synthesis Tool \(Vitis\)](#): Coming from a noticeable FPGA vendor, the tool suite comes along a high level synthesis tool, tightly-coupled to the Xilinx chips and cannot be used for products from other vendors.
- It accepts C, C++, SystemC or openCL API C kernel to generate implementation in RTL format that can be further synthesized to gate-level netlist and FPGA bitstream by the Xilinx tool chain.
- The C-based input is restricted to a synthesizable subset, excluding dynamic memory allocation, recursion, system calls etc.
- The user may guide the synthesis by manually invoking various area and timing optimization directives.
- Exploring various design options using these directives does not alter the source code.

The Current Generation of HLS: Xilinx

- Various powerful directives for iterative code chunks include **loop unrolling**, **loop pipelining**, **flattening a loop hierarchy**, **merging serial loops** together and parallel execution of multiple loops.
- Abstraction of algorithmic descriptions, data type specifications with fixed-point or floating-point integers, and interfaces (FIFO, memories, AXI4)
- Extensive libraries for including **built-in support** for **arbitrary precision data types**, **streams**, and **vectorized data types**.
- **Directive-driven, architecture-aware synthesis**.
- Fast time to QoR that rivals hand-coded RTL
- Accelerated verification using C/C++ test bench simulation and automatic VHDL or Verilog simulation and test bench generation
- Automatic use of AMD on-chip memories and DSP elements, including for floating-point types.

The Current Generation of HLS: Intel

- [Intel High Level Synthesis Compiler](#):
- Intel HLS Compiler is included in the Intel Quartus Prime Design Software Pro Edition installation.
- Uses untimed **ANSI C++** as the golden design source
- Allows to explore multiple architectures through high-level directives
- Supports verification of RTL by comparison with the original C++ source model
- Generates reusable intellectual property (IP) for system integration using the Platform Designer (formerly Qsys)
- Supports inference of streaming, memory mapped, or wire interfaces
- Performs device-specific timing-driven schedule optimization and technology mapping for Intel FPGAs
- Supports a software compiler use model and industry standards including ac_int data types
- Detailed reporting feature for a birds-eye view: High-level design HTML reports are automatically generated during the simulation stage lets users see bottlenecks in their design
- Allows users to view and analyze: Area utilization, loop structure, memory usage, system data flow, clusters, and surrounding logic
- Supports multiple flows to integrate IP in a system. Integrate HLS code through direct HLD instantiation, through Platform Designer

The Current Generation of HLS:

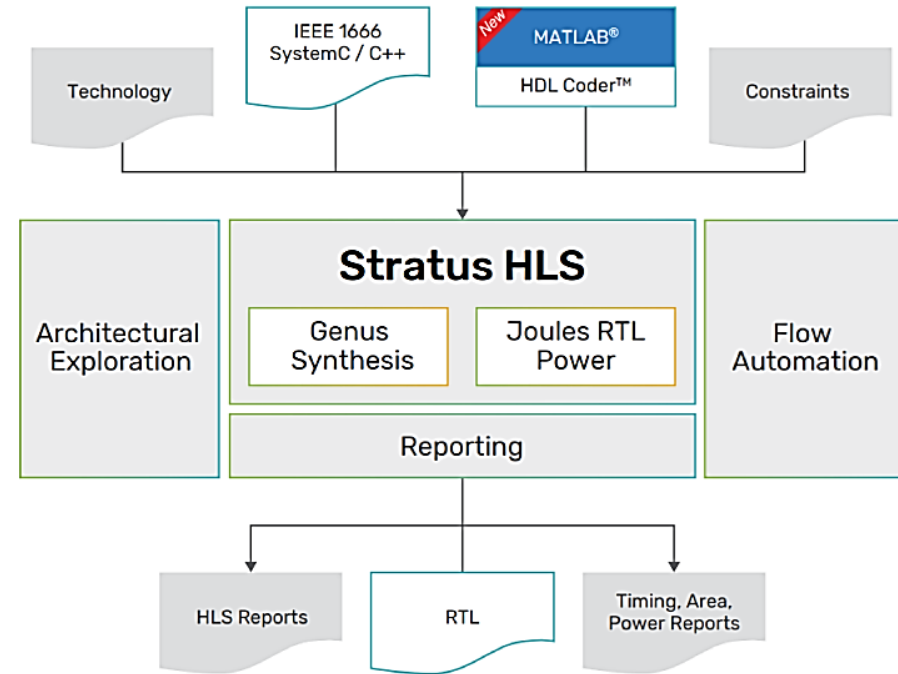
- [Synopsys Symphony C Compiler](#): The high-level synthesis tool from Synopsys, called Symphony C, was formally Synfora's PICO Express, originated after research at HP Labs.
- The tool accepts a broad subset of C/C++ for a fixed-point sequential code that may have loop nesting and multiple levels of hierarchy.
- Its optimization engine efficiently analyzes through multi-level compilation techniques to extract parallelization across the design source.
- The tools carries out optimizations for **resource sharing** by effective **scheduling** and automatic **pipelining** for target-aware timing closure as per the user constraints.
- A **verification environment** with **RTL testbench**, **test vectors**, script and SystemC **verification** model including TLM System, **cycle accurate** wrapper are also generated.
- A seamless integration for FPGAs with synopsys synplify tool and for ASICs with the Design Compiler tool is provided.

The Current Generation of HLS: MathWorks

- [HDL Coder](#): HDL Coder enables high-level design for FPGAs, SoCs, and ASICs by generating portable, synthesizable Verilog, SystemVerilog, and VHDL code from MATLAB functions, Simulink models, and Stateflow charts.
- HDL Coder includes a workflow advisor that automates prototyping generated code on AMD, Intel, and Microchip boards and generates IP cores for ASIC and FPGA workflows.
- User can optimize for speed and area, highlight critical paths, and generate resource utilization estimates before synthesis.
- HDL Coder provides traceability between Simulink models and generated Verilog, SystemVerilog, and VHDL code, enabling code verification for high-integrity applications adhering to DO-254 and other standards.
- Choosing from HDL-ready Simulink blocks and MATLAB functions; add Stateflow charts, Simscape models, and deep learning networks.
- Explore hardware architectures and fixed-point quantization options before committing to an RTL implementation. Use high-level synthesis optimizations such as **resource sharing**, **pipelining**, and **delay balancing**, that efficiently map to device resources such as logic, DSPs, and RAMs.

The Current Generation of HLS: Cadence

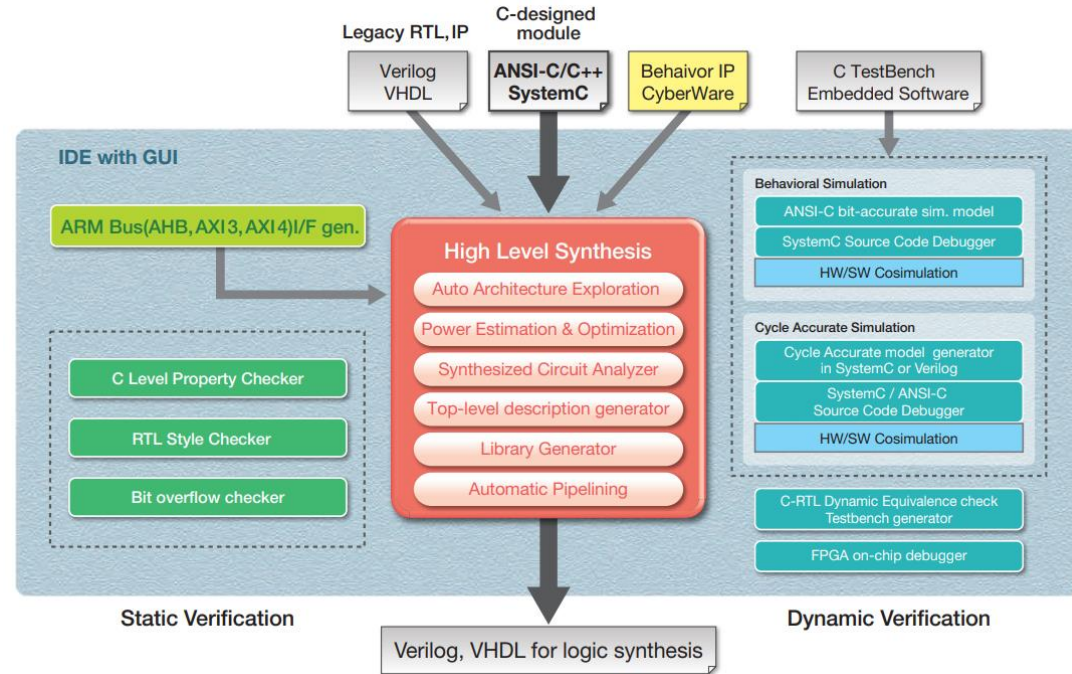
- Cadence Stratus High-Level Synthesis (HLS) automatically creates high-quality register transfer level (RTL) design implementations for ASIC, system-on-chip (SoC), and FPGA targets from high-level IEEE 1666 SystemC, C++, and MATLAB descriptions.
- Supports industry-standard IEEE 1666 SystemC, C, C++, and MATLAB.
- Applicability across both **control-centric** and **datapath-centric** designs.
- Support for hierarchical design and verification, including multiple modules and threads.
- Advanced **pipelining** optimizations support fully pipelined and partially pipelined designs with stalling, draining, and bubble squashing.



https://www.cadence.com/content/dam/cadence-www/global/en_US/images/resources/datasheets/stratus-high-level-synthesis-1.png

The Current Generation of HLS: NEC

- [CyberWorkBench](#): “All-in-C” synthesis and verification.
- Any type of module (controller or datapath) can be synthesized and verified at the original C-level.



The Current Generation of HLS:

- [QCC development platform](#) accepts High Level Language (C/C++/Fortran) as input and generates an optimized, partitioned, and accelerated executable.
- Outputs are supported from the same original source code with no hardware specific languages or pragmas (cuda/opencl/HDL/oneAPI) necessary.
- The platform supports the generation of multithreaded x86, ARM, RISCV or M1 executables, partitioned x86 + nVidia/AMD GPU kernel accelerators, partitioned x86 + FPGA executables targeting FPGA PCIe attached accelerators, partitioned ARM + FPGA executable targeting FPGA SOCs and partitioned RISCV + FPGA executable targeting FPGA SOCs.
- Multithreaded executables are compiled to execute on multicore devices.
- The qcc Development Platform offers a complete suite of analysis tools. Analysis and optimization capabilities include FPGA area estimation, FPGA device selection, FPGA performance estimation, and memory access analysis.

The Current Generation of HLS:

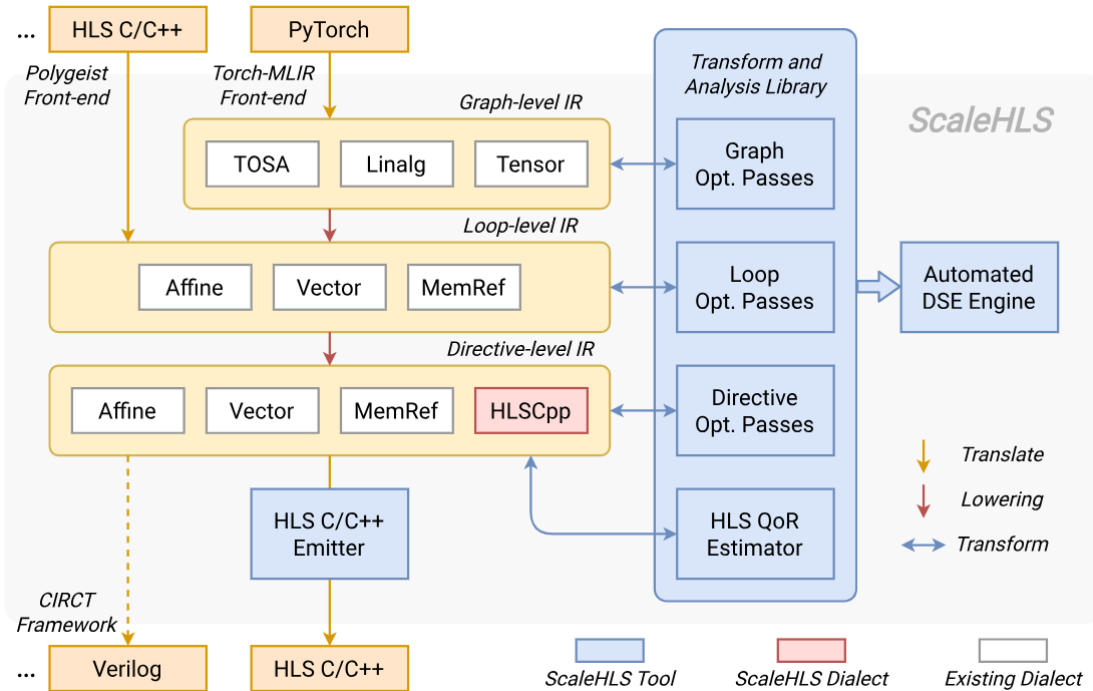
- [Hastlayer](#) targets software developers not having any FPGA design knowledge.
- The general process of utilizing Hastlayer to convert a piece of software into an FPGA-implemented hardware that delivers the same logic is as following:
 1. A performance-critical part of the software is factored out to a separate assembly (class library).
 2. The Hastlayer class library is added to the software and configured.
 3. Method calls to the performance-critical component are changed to go through Hastlayer. This makes it possible to direct calls to the hardware implementation when it's ready but keep the option to run the standard software assembly if e.g. an FPGA is not present.
 4. The conversion of the performance-critical assembly is done by Hastlayer: VHDL code is generated which is put into the Hastlayer hardware framework, then synthesized and used to program a connected FPGA with the existing vendor toolchain.
 5. Method calls are now directed to the FPGA and are executed by logic circuits.
 6. The result of the process is then passed back to the software components calling the methods and then the software continues its execution the regular way.

The Current Generation of HLS:

- Instant SoC (System on Chip) compiles C++ code directly to synthesizable VHDL.
- The VHDL code includes a RISC-V soft CPU, CPU instructions, memories and peripherals. This is the fastest and easiest way to create a CPU system for FPGAs.
- The CPU peripherals like logic IOs, UARTs, SPI, I2C, AXI4 Streams etc are defined as C++ objects.
- The VHDL code is optimized with respect to the generated RISC-V instructions.
- Only used instructions are implemented.
- The memories are optimized using static stack analysis.

The Current Generation of HLS:

- [ScaleHLS](#): ScaleHLS is a High-level Synthesis (HLS) framework on MLIR ([Multi-Level Intermediate Representation](#)).
- MLIR is a compilation framework supporting multiple levels of functional and representational hierarchy.
- ScaleHLS can compile C/C++ or PyTorch model to optimized HLS C/C++ to generate high-efficiency RTL design using downstream tools, such as Xilinx Vivado HLS.



Ye, Hanchen, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. "Scalehls: A new scalable high-level synthesis framework on multi-level intermediate representation." In 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp. 741-755. IEEE, 2022.

The Current Generation of HLS:

- [CIRCT](#) stands for “Circuit IR Compilers and Tools”.
- It is an open-source project that aims to apply MLIR and the LLVM development methodology to the domain of hardware design tools.
- The project provides various circuit languages such as FIRRTL, ESI, and SystemVerilog, and offers tools for verification, synthesis, simulation, and more.
- CIRCT is still an experimental effort, but it has already gained a lot of attention from the hardware design community.
- If you are interested in learning more about CIRCT, you can check out the official website or the GitHub repository.

The Current Generation of HLS:

- [XLS](#) implements a High-Level Synthesis toolchain that produces synthesizable designs (Verilog and SystemVerilog) from flexible, high-level descriptions of functionality. It is Apache 2 licensed.
- XLS attempts to leverage automation, software engineers, and machine cycles to accelerate this overall process.
- XLS supports concurrent processes, in **Communicating Sequential Processes** (CSP) style, that allow pipelines to communicate with each other and induct over time.
- It is still under active development.
- Example: [RISC-V implementation in 500 lines of code](#).

The Current Generation of HLS:

- GAUT: GAUT was an open-source HLS tool from University of South Brittany (Lab-STICC) dedicated to DSP applications.
- It accepts a bit-accurate C/C++ specification of the design and extracts a DFG model by gcc/g++ compiler.
- The mandatory constraints for the design are initial latency and clock period while the optional constraints are memory architecture mapping and I/O timing diagram (interface definition).
- GAUT generates a library of time characterized operators for the HLS steps based on a technological library and technology node specified.
- The scheduling is performed by list scheduling algorithm, followed by a global optimization on interconnects and storage elements.
- GAUT generates a VHDL testbench and scripts for Modelsim as well as SystemC cycle accurate simulation model.



Hardware Description Languages

Introduction to HDLs

- The input to high-level synthesis (HLS) is typically a **behavioral description** written in a **hardware-description language** (HDL).
- HDLs allow the user to specify the design's outputs in terms of the inputs over time using **abstract data** carriers and **language operators**.
- At a glance, such a language might seem similar to a high-level **programming language** like C, C++ or Python, in which programs are written using sequences of statements with expressions that assign values to variables.
- However, the semantics of a behavior specified in an HDL implies a hardware design yet to be implemented (built), rather than a piece of behavior to be executed on an existing machine.
- Furthermore, these HDLs have additional constructs to accommodate the intrinsic features of hardware, such as the notions of clocking, concurrency and asynchrony.

Hardware Description Languages

VHDL: Introduction

- VHSIC (Very High Speed Integrated Circuits) Hardware Description Language.
- In 80's designed by IBM, Texas Instruments, and Intermetrics as part of the DoD funded VHSIC program.
- Standardized by the IEEE in 1987: IEEE 1076-1987
- Enhanced version of the language defined in 1993: IEEE 1076-1993
- Additional standardized packages provide definitions of data types and expressions of timing data
- IEEE 1164 (data types)
- IEEE 1076.3 (numeric)
- IEEE 1076.4 (timing)

VHDL: Introduction (contd.)

- Initial objective was modelling only and thus only a simulator was envisaged.
- Subsequently tools for VHDL synthesis were developed.
- Object Oriented methodology for you C people can be observed.
- Modules can be used and reused.
- VHDL is a programming language that allows one to model and develop complex digital systems in a dynamic environment.
- Allows you to designate in/out ports (bits) and specify behavior or response of the system.

VHDL: introduction (contd.)

→ Why VHDL ?

→ Public Availability

- Developed initiated under Government Contract

- Now is an IEEE standard

→ Design Methodology and Design Technology Support

→ Technology and Process Independent

→ Wide Range of Descriptive capabilities

- Digital System (e.g. box) level to gate level

- Capability of mixing descriptions

→ Design Exchange

→ Large Scale Design and Design re-use

Simulation Fundamentals

- Purpose of simulation is to verify the behavior of a system by applying stimulation at the inputs and monitoring the response of the system over a period of time.
- There are three types of simulators:
 - Purely analog simulator.
 - A simulator with both digital and analog simulation capabilities.
 - Purely digital simulator.
- In digital simulation only logic level of the measured quantity is determined; no precise value is required.
- In analog simulation the precise values of the measured quantities are determined

Simulation Fundamentals (contd.)

- The system is described with a Hardware Description Language (HDL). The design contains a number of concurrently operating blocks connected with each other by signals.
- Maintains node values at logic level.
- Maintains a time wheel to model propagation of time.
- Evaluates circuit behavior at intervals of time. The interval is chosen as the smallest unit of time after which a node can change its state.

Modeling Hardware

- The VHDL Language
 - A language for describing digital and analog systems.
 - Makes no assumptions about the technology.
 - Multiple levels of abstraction for modeling
 - Behavioral
 - Structural
 - Dataflow
 - Behavioral model, timing model and structural model are integrated.
 - A VHDL process models a block and a VHDL signal models the connection between different blocks.

Structural Model

- Digital circuits consist of components and interconnection between them
- A component can in turn be composed of sub-components and their interconnections
- A component interacts with other components through pins
- Component is modeled as entity
- Component pins are modeled as ports
- Interconnections between components are modeled as signals

Behavioral Model

- The behavior of a component is modeled inside an architecture body of the entity
- It may be described using a collection of concurrently executing statements
- A concurrent statement is sensitive to a set of input signals and is executed whenever any of its sensitive signal changes its value
- A concurrent statement called process statement can contain one or more sequential statements
- A set of sequential statements can be clubbed together in a subprogram

Dataflow Model

- The flow of data through the entity is modelled primarily using concurrent signal assignment statements.
- The structure of the entity is not explicitly specified but it can be implicitly deduced.

Architecture MYARCH of MYENT is
begin

SUM <= A xor B after 8ns

end MYARCH;

VHDL Basics

Basic VHDL Concepts

- Interfaces -- i.e. ports
- Behavior
- Structure
- Test Benches
- Analysis, simulation
- Synthesis

Modeling Styles:

- Semantic model of VHDL
- Structural description
- Data Flow description
- Algorithmic description
- RTL description

1 VHDL Through Examples: Combinational logic using VHDL gate models

VHDL Basics

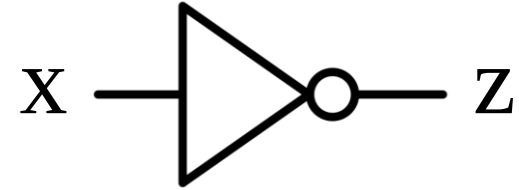
→ A general structure for a VHDL description of a unit

```
entity component is
  port(in_1,in_2,...,in_m      : in  type;
       out_1,out_2,...,out_n  : out type;
       io_1,io_2,...,io_k     : inout type);
end component ;

architecture abstraction of component is
begin
  ...
  ...
  ...
end abstraction ;
```

VHDL Through Examples

→ NOT gate

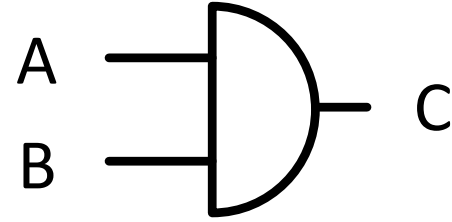


```
entity Not1 is
    port (x : in BIT; z: out BIT);
end entity Not1;

architecture ex1 of Not1 is
begin
    z <= not x;
end architecture ex1;
```

VHDL Through Examples

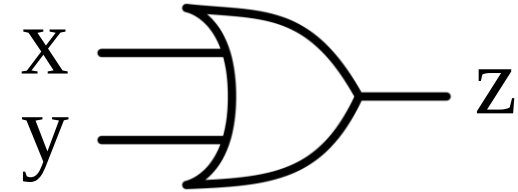
→ Two input AND gate



```
1 entity AND2 is
2     port (A,B: in bit; C: out bit);
3 end AND2;
4
5 architecture beh of AND2 is
6 begin
7     C <= A and B;
8 end beh;
```

VHDL Through Examples

→ Two input OR gate

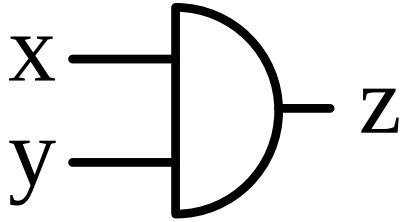


```
entity Or2 is
    port (x, y : in BIT; z: out BIT);
end entity Or2;

architecture ex1 of Or2 is
begin
    z <= x or y;
end architecture ex1;
```

VHDL Through Examples

- Two input AND gate,
Different description

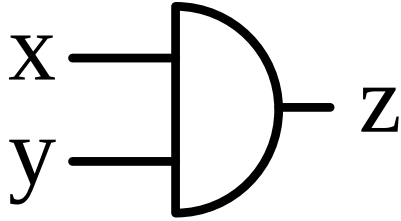


```
entity And2 is
    port (x, y : in BIT; z: out BIT);
end entity And2;

architecture ex3 of And2 is
    signal xy : BIT_VECTOR(0 to 1);
begin
    xy <= x&y;
    with xy select
        z <= '1' when "11",
            '0' when others;
end architecture ex3;
```

VHDL Through Examples

→ Two input AND gate, with delay



```
entity And2 is
    generic (delay : DELAY_LENGTH);
    port (x, y : in BIT; z: out BIT);
end entity And2;

architecture ex2 of And2 is
begin
    z <= x and y after delay;
end architecture ex2;
```

VHDL Through Examples

→ Three input NAND with delay

```
entity Nand3 is
    port (w, x, y : in BIT; z: out BIT);
end entity Nand3;

architecture ex1 of Nand3 is
begin
    z <= not (w and x and y) after 5 NS;
end architecture ex1;
```

VHDL Through Examples

→ Logic function:

$$\rightarrow a = (y \cdot (x \oplus \text{invert})) + (\text{invert} \cdot y')$$

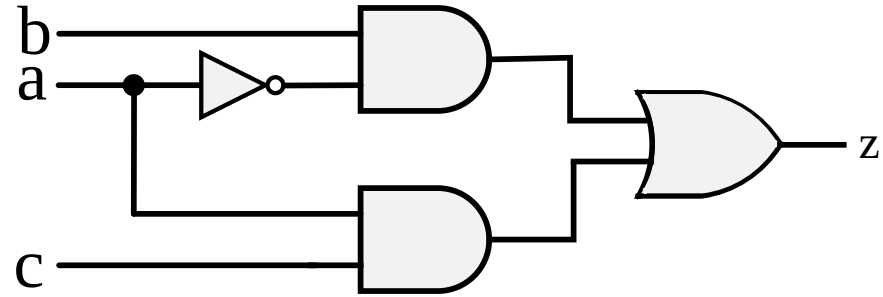
$$\rightarrow o = (x' \cdot (y \oplus \text{invert})) + (\text{invert}' \cdot x)$$

```
entity universal is
    port (x, y, invert : in BIT; a, o : out BIT);
end entity universal;

architecture univ of universal is
begin
    a <= (y and (x xor invert)) or (invert and not y);
    o <= (not x and (y xor invert)) or (x and not invert);
end architecture univ;
```

VHDL Through Examples

→ A Comb function.

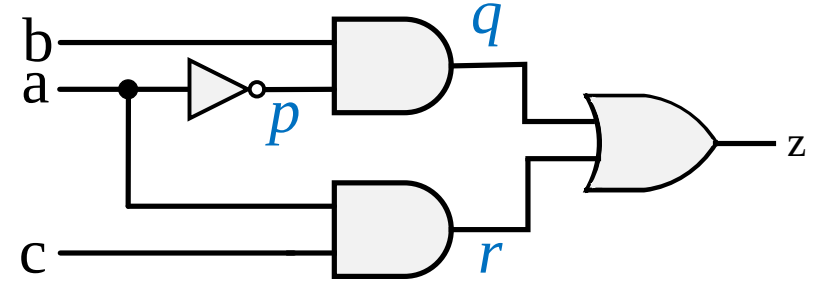


```
entity comb_function is
    port (a, b, c : in BIT; z: out BIT);
end entity comb_function;

architecture expression of comb_function is
begin
    z <= (not a and b) or (a and c);
end architecture expression;
```

VHDL Through Examples

- A Comb function, using sub modules (a two layer hierarchical design)



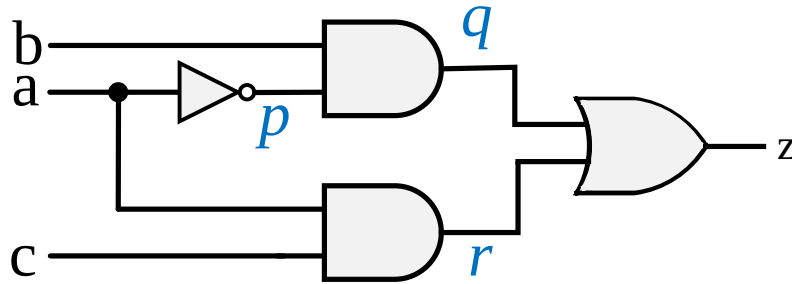
```
architecture netlist of comb_function is

    signal p, q, r : BIT;

begin
    g1: entity WORK.Not1(ex1) port map (a, p);
    g2: entity WORK.And2(ex1) port map (p, b, q);
    g3: entity WORK.And2(ex1) port map (a, c, r);
    g4: entity WORK.Or2(ex1) port map (q, r, z);
end architecture netlist;
```

VHDL Through Examples

→ A Comb function, using components (a two layer hierarchical design), a different description.



```
architecture netlist2 of comb_function is
    component And2 is
        port (x, y : in BIT; z: out BIT);
    end component And2;

    component Or2 is
        port (x, y : in BIT; z: out BIT);
    end component Or2;

    component Not1 is
        port (x : in BIT; z: out BIT);
    end component Not1;

    signal p, q, r : BIT;

begin
    g1: Not1 port map (a, p);
    g2: And2 port map (p, b, q);
    g3: And2 port map (a, c, r);
    g4: Or2 port map (q, r, z);
end architecture netlist;
```

VHDL Through Examples

- A Full-Adder, described using sub modules (a two layer hierarchical design).

```
entity FullAdder is
    port (a, b, Ci : in BIT; S, Co: out BIT);
end entity FullAdder;

architecture netlist of FullAdder is
    signal na, nb, nc, d, e, f, g, h, i, j : BIT;
begin
    n0 : entity WORK.Not1 port map (a, na);
    n1 : entity WORK.Not1 port map (b, nb);
    n2 : entity WORK.Not1 port map (Ci, nc);
    a0 : entity WORK.And3 port map (na, nb, Ci, d);
    a1 : entity WORK.And3 port map (na, b, nc, e);
    a2 : entity WORK.And3 port map (a, b, Ci, f);
    a3 : entity WORK.And3 port map (a, nb, nc, g);
    o0 : entity WORK.Or4 port map (d, e, f, g, S);
    a4 : entity WORK.And2 port map (b, Ci, h);
    a5 : entity WORK.And2 port map (a, b, i);
    a6 : entity WORK.And2 port map (a, Ci, j);
    o1 : entity WORK.Or3 port map (h, i, j, Co);
end architecture netlist;
```

VHDL Through Examples

→ A testbench for Full-Adder.

```
entity testadder is
end entity testadder;

architecture ta of testadder is
    signal a, b, Ci, S, Co : BIT;
begin
    f0 : entity WORK.FullAdder port map(a, b, Ci, S, Co);
    a <= '0', '1' after 10 NS, '0' after 20 NS, '1' after 30 NS,
        '0' after 40 NS, '1' after 50 NS, '0' after 60 NS,
        '1' after 70 NS;
    b <= '0', '1' after 20 NS, '0' after 40 NS, '1' after 60 NS;
    Ci <= '0', '1' after 40 NS;
end architecture ta;
```

VHDL Through Examples

→ A testbench for AND-2

```
entity TestAnd2 is
end entity TestAnd2;

architecture alternate of TestAnd2 is
  component A2 is
    port (x, y : in BIT; z: out BIT);
  end component A2;
  for all : A2 use entity work.And2(ex2);
  signal a,b,c : BIT;
begin
  g1: A2 port map (x=>a, y=>b, z=>c);
  a<= '0', '1' after 100 NS;
  b<= '0', '1' after 150 NS;
end architecture alternate;
```

```
entity And2 is
  generic (delay : DELAY_LENGTH);
  port (x, y : in BIT; z: out BIT);
end entity And2;

architecture ex2 of And2 is
begin
  z <= x and y after delay;
end architecture ex2;
```

VHDL Through Examples

→ Another testbench for AND-2

```
entity TestAnd2 is
end entity TestAnd2;

architecture io of TestAnd2 is

    signal a,b,c : BIT;

begin
    g1: entity WORK.And2(ex2) port map (x=>a, y=>b, z=>c);
    a<= '0', '1' after 100 NS;
    b<= '0', '1' after 150 NS;
end architecture io;
```

```
entity And2 is
    generic (delay : DELAY_LENGTH);
    port (x, y : in BIT; z: out BIT);
end entity And2;

architecture ex2 of And2 is
begin
    z <= x and y after delay;
end architecture ex2;
```

2

VHDL Through Examples: Combinational Building Blocks

VHDL Through Examples

→ Tri-State buffer, two architectures.

```
library IEEE;
use IEEE.std_logic_1164.all;
entity three_state is
  port (a, enable : in std_logic;
        z : out std_logic);
end entity three_state;

architecture when_else of three_state is
begin
  z <= a when enable = '1' else 'Z';
end architecture when_else;

architecture after_when_else of three_state is
begin
  z <= a after 4 NS when enable = '1' else 'Z';
end architecture after_when_else;
```

VHDL Through Examples

→ Different descriptions for MUX-2.

```
library IEEE;
use IEEE.std_logic_1164.all;
entity mux is
  port (a, b, c, d: in std_logic;
        s: in std_logic_vector(1 downto 0);
        y: out std_logic);
end entity mux;
```

```
architecture three_state of mux is
begin
  y <= a when s="00" else 'Z';
  y <= b when s="01" else 'Z';
  y <= c when s="10" else 'Z';
  y <= d when s="11" else 'Z';
end architecture three_state;
```

```
architecture mux1 of mux is
begin
  with s select
    y <= a when "00",
         b when "01",
         c when "10",
         d when "11",
         'X' when others;
end architecture mux1;
```

```
architecture mux2 of mux is
begin
  y <= a when s="00" else
    b when s="01" else
    c when s="10" else
    d when s="11" else
    'X';
end architecture mux2;
```

VHDL Through Examples

- Different descriptions for MUX-2

```
library IEEE;
use IEEE.std_logic_1164.all;
entity mux is
  port (a, b, c, d: in std_logic;
        s: in std_logic_vector(1 downto 0);
        y: out std_logic);
end entity mux;

architecture three_state of mux is
begin
  y <= a when s="00" else 'Z';
  y <= b when s="01" else 'Z';
  y <= c when s="10" else 'Z';
  y <= d when s="11" else 'Z';
end architecture three_state;
```

VHDL Through Examples

N-bit Adder, functional model:

- Unsigned Adder
- Signed Adder

```
library IEEE;
use IEEE.std_logic_1164.all, IEEE.numeric_std.all;

entity NBitAdder is
    generic (n: NATURAL :=4);
    port(A, B: in std_logic_vector(n-1 downto 0);
         Cin : in std_logic;
         Sum : out std_logic_vector(n-1 downto 0);
         Cout: out std_logic);
end entity NBitAdder;

-- Unsigned Adder
architecture unsigned of NBitAdder is
    signal result : unsigned(n downto 0);
    signal carry : unsigned(n downto 0);
    constant zeros : unsigned(n-1 downto 0) := (others => '0');
begin
    carry <= (zeros & Cin);
    result <= ('0' & unsigned(A)) + ('0' & unsigned(B)) + carry;
    Sum <= std_logic_vector(result(n-1 downto 0));
    Cout <= result(n);
end architecture unsigned;

-- Signed Adder
architecture sgned of NBitAdder is
    signal result : signed(n downto 0);
    signal carry : signed(n downto 0);
    constant zeros : signed(n-1 downto 0) := (others => '0');
begin
    carry <= (zeros & Cin);
    result <= (A(n-1) & signed(A)) + (B(n-1) & signed(B)) + carry;
    Sum <= std_logic_vector(result(n-1 downto 0));
    Cout <= result(n);
end architecture sgned;
```

VHDL Through Examples

→ n-bit Adder, a ripple carry.

```
architecture StructIterative of NBitAdder is
    signal Carry: std_logic_vector(0 to n);
begin
    g1 : for i in 0 to n-1 generate
        lt : if i = 0 generate
            f0 : entity work.FullAdder port map (A(i), B(i), Cin,
                                                    Sum(i), Carry(i+1));
        end generate lt;
        rt : if i = n-1 generate
            fn : entity work.FullAdder port map (A(i), B(i), Carry(i),
                                                    Sum(i), Cout);
        end generate rt;
        md : if i > 0 and i < n-1 generate
            fm : entity work.FullAdder port map (A(i), B(i), Carry(i),
                                                    Sum(i), Carry(i+1));
        end generate md;
    end generate g1;
end architecture StructIterative;
```

VHDL Through Examples

- Testbench for n-bit Adder

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;
entity TestNBitAdder is
end entity TestNBitAdder;

architecture TestBench_1 of TestNBitAdder is
    constant n: NATURAL := 4;
    signal A, B, Sum: std_logic_vector (n-1 downto 0);
    signal Cin, Cout: std_logic;
begin
    s0: entity WORK.NBitAdder(unsnged) generic map (n)
        port map(A, B, Cin, Sum, Cout);
    Cin <= '0', '1' after 10 NS, '0' after 25 NS;
    A <= "0000", "1111" after 5 NS, "0111" after 15 NS;
    B <= "0000", "1111" after 20 NS;
end architecture TestBench_1;

architecture TestBench_2 of TestNBitAdder is
    constant n: NATURAL := 4;
    signal A, B, Sum: std_logic_vector (n-1 downto 0);
    signal Cin, Cout: std_logic;
begin
    s0: entity WORK.NBitAdder(unsnged) generic map (n)
        port map(A, B, Cin, Sum, Cout);
```

VHDL Through Examples

- Comparator, a sequential description

```
library IEEE;
use IEEE.std_logic_1164.all;
entity comparator is
    port (x, y : in std_logic_vector;
          eq : out std_logic);
end entity comparator;

architecture iterative of comparator is
begin
    process (x,y) is
        variable eqi : std_logic;
    begin
        eqi := '1';
        for i in x'RANGE loop
            eqi := eqi and (x(i) xnor y(i));
        end loop;
        eq <= eqi;
    end process;
end architecture iterative;
```

VHDL Through Examples

2-4 Decoder, with different descriptions:

- Using WHEN ELSE
- Using WITH SELECT

```
library IEEE;
use IEEE.std_logic_1164.all;

entity decoder is
    port (a : in std_logic_vector(1 downto 0);
          z : out std_logic_vector(3 downto 0));
end entity decoder;

architecture when_else of decoder is
begin
    z <= "0001" when a = "00" else
         "0010" when a = "01" else
         "0100" when a = "10" else
         "1000" when a = "11" else
         "XXXX";
end architecture when_else;

architecture with_select of decoder is
begin
    with a select
        z <= "0001" when "00",
             "0010" when "01",
             "0100" when "10",
             "1000" when "11",
             "XXXX" when others;
end architecture with_select;
```

VHDL Through Examples

- A 3-to-8 Decoder, using Boolean expressions.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity decoder is
    port (a : in std_logic_vector(2 downto 0);
          z : out std_logic_vector(7 downto 0));
end entity decoder;

architecture bool_expr of decoder is
begin
    z(0) <= not a(0) and not a(1) and not a(2);
    z(1) <= a(0) and not a(1) and not a(2);
    z(2) <= not a(0) and a(1) and not a(2);
    z(3) <= a(0) and a(1) and not a(2);
    z(4) <= not a(0) and not a(1) and a(2);
    z(5) <= a(0) and not a(1) and a(2);
    z(6) <= not a(0) and a(1) and a(2);
    z(7) <= a(0) and a(1) and a(2);
end architecture bool_expr;
```

VHDL Through Examples

3-8 Decoder, different descriptions:

- Using WHEN ELSE
- Using WITH SELECT

```
architecture when_else of decoder is
begin
    z <= "00000001" when a = "000" else
        "00000010" when a = "001" else
        "00000100" when a = "010" else
        "00001000" when a = "011" else
        "00010000" when a = "100" else
        "00100000" when a = "101" else
        "01000000" when a = "110" else
        "10000000" when a = "111" else
        "XXXXXXXXX";
end architecture when_else;

-----

architecture with_select of decoder is
begin
    with a select
        z <= "00000001" when "000",
            "00000010" when "001",
            "00000100" when "010",
            "00001000" when "011",
            "00010000" when "100",
            "00100000" when "101",
            "01000000" when "110",
            "10000000" when "111",
            "XXXXXXXXX" when others;
end architecture with_select;
```

VHDL Through Examples

→ Testbench for
2-4 Decoders

```
library IEEE;
use IEEE.std_logic_1164.all;
entity testdecode is
end entity testdecode;

architecture td of testdecode is
    signal a : std_logic_vector(2 downto 0);
    signal z0, z1, z2 : std_logic_vector(7 downto 0);
    for d0 : decoder use entity work.decoder(bool_expr);
    for d1 : decoder use entity work.decoder(when_else);
    for d2 : decoder use entity work.decoder(with_select);

begin
    d0 : entity WORK.decoder(bool_expr) port map (a, z0);
    d1 : entity WORK.decoder(when_else) port map (a, z1);
    d2 : entity WORK.decoder(with_select) port map (a, z2);
    a <= "000", "001" after 10 NS, "010" after 20 NS;
end architecture td;
```

VHDL Through Examples

→ n-to-2ⁿ Decoder

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;
entity decoder is
    generic (n : POSITIVE);
    port (a : in std_logic_vector(n-1 downto 0);
          z : out std_logic_vector(2**n-1 downto 0));
end entity decoder;

architecture rotate of decoder is
    constant z_out : BIT_VECTOR(2**n-1 downto 0) :=
        (0 => '1', others => '0');
begin
    z <= to_StdLogicVector(z_out sll to_integer(unsigned(a)));
end architecture rotate;
```

VHDL Through Examples

➤ Priority 4-to-2 Encoder

```
library IEEE;
use IEEE.std_logic_1164.all;
entity priority is
    port (a: in std_logic_vector(3 downto 0);
          y: out std_logic_vector(1 downto 0);
          valid: out std_logic);
end entity priority;

architecture DontCare of priority is
begin
    with a select
        y <= "00" when "0001",
              "01" when "001-",
              "10" when "01--",
              "11" when "1---",
              "00" when others;
        valid <= '1' when a(0)='1' or a(1)='1' or a(2)='1' or a(3) = '1'
              else '0';
end architecture DontCare;
```

VHDL Through Examples

Priority

4-to-2 Encoder,

Different
descriptions

```
architecture Ordered of priority is
begin
    y <= "11" when a(3)='1' else
         "10" when a(2)='1' else
         "01" when a(1)='1' else
         "00" when a(0)='1' else
         "00";
    valid <= '1' when a(0)='1' or a(1)='1' or a(2)='1' or a(3) = '1'
             else '0';
end architecture Ordered;
```

```
use IEEE.numeric_std.all;
architecture Match of priority is
begin
    y <= "00" when std_match(a, "0001") else
         "01" when std_match(a, "001-") else
         "10" when std_match(a, "01--") else
         "11" when std_match(a, "1---") else
         "00";
    valid <= '1' when a(0)='1' or a(1)='1' or a(2)='1' or a(3) = '1'
             else '0';
end architecture Match;
```

VHDL Through Examples

- 4-to-2 Priority Encoder
- A sequential description

```
architecture Sequential of priority is
begin
    process (a) is
    begin
        if a(3)='1' then
            y <= "11";
            valid <= '1';
        elsif a(2)='1' then
            y <= "10";
            valid <= '1';
        elsif a(1)='1' then
            y <= "01";
            valid <= '1';
        elsif a(0)='1' then
            y <= "00";
            valid <= '1';
        else
            y <= "00";
            valid <= '0';
        end if;
    end process;
end architecture Sequential;
```

VHDL Through Examples

- 4-to-2 Priority Encoder
- A sequential description

```
architecture Sequential2 of priority is
begin
  process (a) is
  begin
    valid <= '1';
    if a(3)='1' then
      y <= "11";
    elsif a(2)='1' then
      y <= "10";
    elsif a(1)='1' then
      y <= "01";
    elsif a(0)='1' then
      y <= "00";
    else
      valid <= '0';
      y <= "00";
    end if;
  end process;
end architecture Sequential2;
```

VHDL Through Examples

➤ 2^n -n priority encoder

```
library IEEE;
use IEEE.std_logic_1164.all, IEEE.numeric_std.all;
entity priority is
    generic (n : POSITIVE);
    port (a: in std_logic_vector(2**n-1 downto 0);
          y: out std_logic_vector(n-1 downto 0);
          valid: out std_logic);
end entity priority;

architecture iterative of priority is
begin
    process (a) is
    begin
        valid <= '0';
        y <= (others => '0');
        for i in a'RANGE loop
            if a(i) = '1' then
                y <= std_logic_vector(to_unsigned(i, n));
                valid <= '1';
                exit;
            end if;
        end loop;
    end process;
end architecture iterative;
```

VHDL Through Examples

- Parity checker.
- An iterative description

```
library IEEE;
use IEEE.std_logic_1164.all;
entity parity is
    port (a : in std_logic_vector;
          y : out std_logic);
end entity parity;

architecture iterative of parity is
begin
    process (a) is
        variable even : std_logic;
    begin
        even := '0';
        for i in a'RANGE loop
            if a(i) = '1' then
                even := not even;
            end if;
        end loop;
        y <= even;
    end process;
end architecture iterative;
```

VHDL Through Examples

BCD to 7-seg decoder

```
library IEEE;
use IEEE.std_logic_1164.all;
entity seven_seg is
    port (a : in std_logic_vector(3 downto 0);
          z : out std_logic_vector(6 downto 0));
end entity seven_seg;

architecture with_select of seven_seg is
begin
    with a select
        z <= "1110111" when "0000",
              "0010010" when "0001",
              "1011101" when "0010",
              "1011011" when "0011",
              "0111010" when "0100",
              "1101011" when "0101",
              "1101111" when "0110",
              "1010010" when "0111",
              "1111111" when "1000",
              "1111011" when "1001",
              "1101101" when "1010",
              "1101101" when "1011",
              "1101101" when "1100",
              "1101101" when "1101",
              "1111011" when "1110",
              "1111111" when "1111",
              "0000000" when others;
end architecture with_select;
```

VHDL Through Examples

- Testbench for n-bit Adder

```
architecture TestBench_2 of TestNBitAdder is
    constant n: NATURAL := 4;
    signal A, B, Sum: std_logic_vector (n-1 downto 0);
    signal Cin, Cout: std_logic;
begin
    s0: entity WORK.NBitAdder(unsignd) generic map (n)
        port map(A, B, Cin, Sum, Cout);

    process is
    begin
        Cin <= '0';
        A <= "0000";
        B <= "0000";
        wait for 5 NS;
        A <= "1111";
        wait for 5 NS;
        Cin <= '1';
        wait for 5 NS;
        A <= "0111";
        wait for 5 NS;
        B <= "1111";
        wait for 5 NS;
        Cin <= '0';
        wait;
    end process;
end architecture TestBench_2;
```

VHDL Through Examples

■ Testbench for n-bit Adder

```
architecture TestBench_3 of TestNBitAdder is
    constant n: NATURAL := 4;
    signal A, B, Sumint : NATURAL;
    signal Aslv, Bslv, Sum: std_logic_vector (n-1 downto 0);
    signal Cin, Cout: std_logic;
begin
    s0: entity WORK.NBitAdder(unsignd) generic map (n)
        port map(Aslv, Bslv, Cin, Sum, Cout);
        Aslv <= std_logic_vector(to_unsigned(A, n));
        Bslv <= std_logic_vector(to_unsigned(B, n));
        Sumint <= to_integer(unsigned(Cout & Sum));
    stim: process is
    begin
        Cin <= '0';
        A <= 0;
        B <= 0;
        wait for 5 NS;
        A <= 15;
        wait for 5 NS;
        Cin <= '1';
        wait for 5 NS;
        A <= 7;
        wait for 5 NS;
        B <= 15;
        wait for 5 NS;
        Cin <= '0';
        wait;
    end process;
end architecture TestBench_3;
```

3

VHDL Through Examples: Synchronous Sequential Design

VHDL Through Examples

→ Two input XOR gate, described using “process”.

```
1  entity Xor_gate  is
2      port (in1, in2 : in bit; Out1 : out bit);
3  end Xor_gate ;
4  architecture behavioral of Xor_gate is
5  begin
6      process
7      begin
8          Out1 <= in1 xor in2;
9          wait on in1, in2;
10     end process;
11 end behavioral;
```

```

entity state_machine is
    port(x, clock, reset :in BIT;
         z :out BIT);
end entity state_machine;

architecture behaviour of state_machine is
    type state_type is (S0, S1, S2);
    signal state, next_state : state_type;

    seq: process (clock, reset) is
    begin
        if reset = '1' then
            state <= S0;
        elsif rising_edge(clock) then
            state <= next_state;
        end if;
    end process seq;

```

```

com: process (state, x) is
begin
    Z <= '0';
    case state is
        when S0 =>
            if X = '0' then
                next_state <= S0;
            else
                next_state <= S1;
            end if;
        when S1 =>
            if X = '0' then
                next_state <= S0;
            else
                next_state <= S2;
            end if;
        when S2 =>
            if X = '0' then
                next_state <= S0;
            else
                Z <= '1';
                next_state <= S2;
            end if;
    end case;
end process com;
end architecture behaviour;

```

- Behavioral Description of FSM

VHDL Through Examples

VHDL Through Examples

```
library IEEE;
use IEEE.std_logic_1164.all;
entity testparity is
end entity testparity;

architecture testbench of testparity is
    signal clock, reset, a, error : std_logic;

begin

    p0: entity WORK.parity port map (clock, reset, a, error);

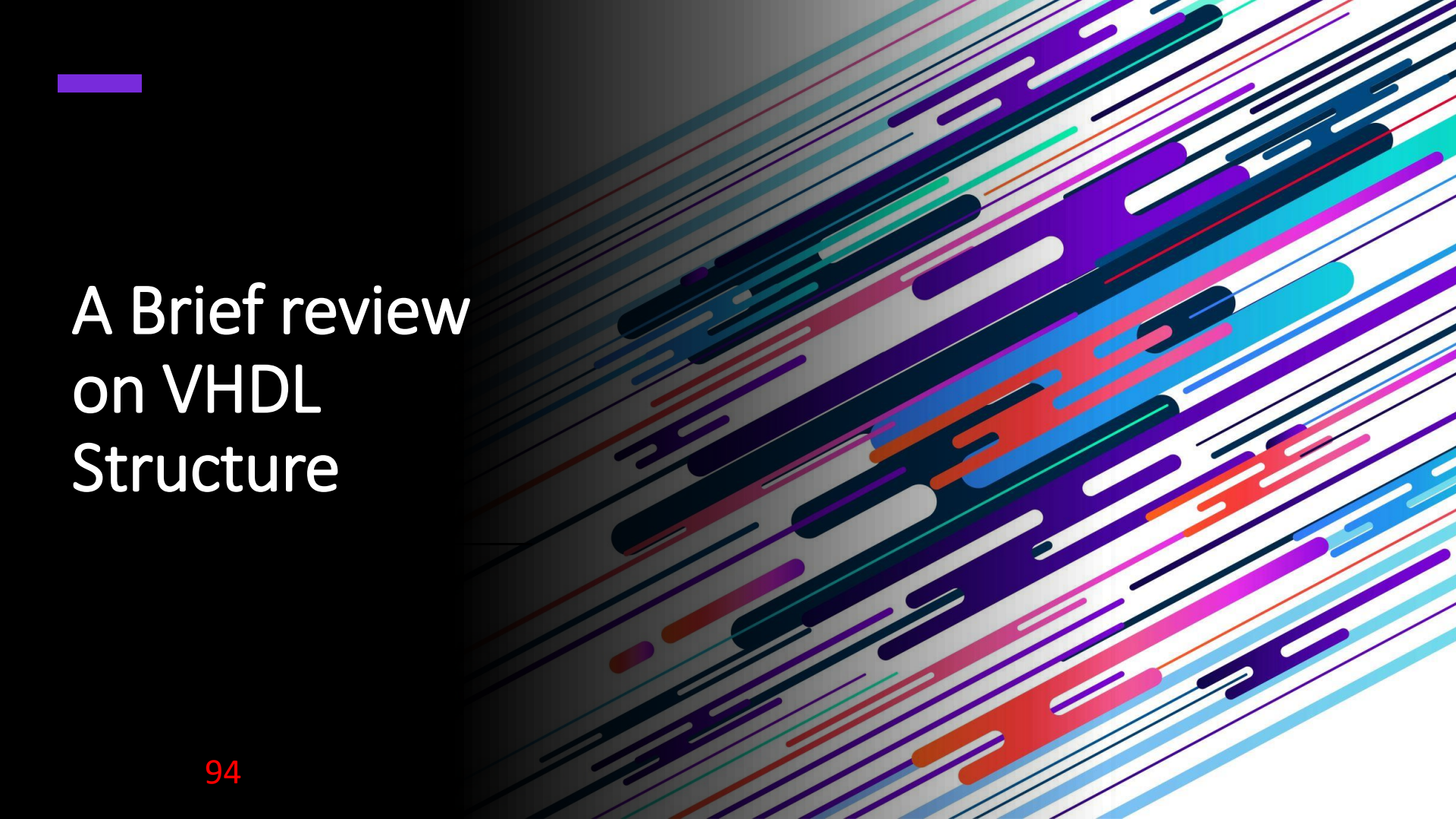
    clk: process is
    begin
        clock <= '0';
        wait for 10 NS;
        clock <= '1';
        wait for 10 NS;
    end process clk;

    res: process is
    begin
        reset <= '0';
        wait for 3 NS;
        reset <= '1';
        wait for 3 NS;
        reset <= '0';
        wait;
    end process res;

    inp: process is
    begin
        a <= '1';
        wait for 60 NS;
        a <= '0';
        wait for 20 NS;
        a <= '1';
        wait;
    end process inp;
end architecture testbench;
```

VHDL Through Examples

VHDL Through Examples



A Brief review on VHDL Structure

VHDL Libraries and Design Units

- A VHDL library is a host dependent storage facility for intermediate-form representations of analyzed design units
- A design unit is a VHDL construction that can be independently analyzed and stored in a design library. A design unit may be a primary or a secondary one.
- Primary design unit
 - **entity** declaration, **package** declaration and **configuration** declaration
- Secondary design unit
 - **architecture** body and **package** body
- In a library, there can be only one primary unit of same name but there can be multiple secondary units by same name
- A secondary unit can have name same as primary unit

Building Blocks in VHDL

- **Entity** Declaration
 - generic, port, declarations, statements
- **Architecture** Body
 - declarations, statements
- Subprogram Declaration
 - parameter - list
- Subprogram Specification
 - declarations statements
- **Package** Declarations
 - declarations
- Package Bodies
 - declarations, subprogram body

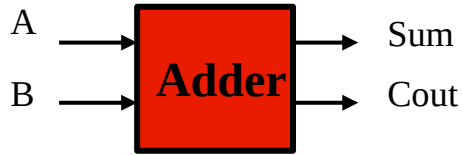
ENTITY

- provides a name to the component
- contains the port definitions in the interface list
- can contain some generic definitions which can be used to override default values

```
entity identifier is
    generic interface_list;
    port    interface_list;
    -- declarations
begin
    -- statements
end entity identifier;
```

Simple Example 3

→ Half Adder *entity*



```
entity Adder is
port(A    : in  Bit;
      B    : in  Bit;
      Sum  : out Bit;
      Cout : out Bit);
end Adder;
```

```
architecture Behavioral_Desc of Adder is
begin
    process (A,B)
        Sum  <= A or  B after 5 ns;
        Cout <= A and B after 6 ns;
    end process;
end Behavioral_Desc
```

Architecture

- Encapsulates the **behavior** and **timing** information
- Contains a number of **concurrent statements**
- There can be **multiple architecture** bodies for a given entity

```
architecture identifier of entity_name is
    -- declarations
begin
    -- statements
end architecture identifier;
```

Simple Example 4

- Half Adder *architecture*
(behavioral)

```
entity Adder is
port(A      : in  Bit;
      B      : in  Bit;
      Sum    : out Bit;
      Cout   : out Bit);
end Adder;
```

```
architecture Behavioral_Desc of Adder is
begin
    process (A,B)
        Sum  <= A or B after 5 ns;
        Cout <= A and B after 6 ns;
    end process;
end Behavioral_Desc
```

Simple Example 5

→ Half Adder *architecture* (structural)

```
architecture Struct_Desc of Adder is
-- declarations
    component Or_Comp
        port(X:in Bit; Y:in Bit; Z:out Bit);
    end component;
    component And_Comp
        port(X:in Bit; Y:in Bit; Z:out Bit);
    end component;
begin
-- component instantiations
    A1: Or_Comp port_map(X => A, y => B, Z => SUM);
    B1: And_Comp port_map(X => A, y => B, Z => Cout);
end Struct_Desc;
```

Subprograms

- Subprograms are of two types :
 - functions and procedures
- A subprogram consists of a sequence of declarations and statements which can be repeated from different locations in VHDL descriptions
- Subprograms can be overloaded
- Functions can be used for operator overloading
- Procedures can assign values to its parameter objects while functions can not
- A subprogram can be separated into its **subprogram declaration** and **subprogram body**

Functions

- Intended to be used strictly for computing values and not for changing value of any objects associated with the function's formal parameters
- All parameters must be of mode *in* and class *signal* or *constant* or *File*.
- If no mode is specified, the parameter is interpreted as having mode *in*. If no class is specified parameters are interpreted as being of class *constant*. Parameter of type FILE has no mode.
- Examples of function declaration
 - Object class of parameter is implicit
function **Mod_256** (**X : Integer**) return **Byte**;
 - Object class of parameter is explicit
function **Mod_256**(constant **X : in Integer**) return **Byte**;

Simple Example 6

→ A function

```
1  -- Function declaration
2  function Min(X,Y : Integer) return Integer;
3
4  -- Function Specification
5  function Min (X,Y : Integer) return Integer is
6  begin
7      if (X < Y) then
8          return X;
9      else
10         return Y;
11     end if;
12 end Min;
```

Procedures

- Procedures are allowed to change the values of the objects associated with its formal parameters
- Parameters of procedures may of mode *in*, *out* or *inout*
- If no mode is specified, the parameter is interpreted as having mode *in*. If no class is specified parameters of mode *in* are interpreted as being of class *constant* and parameters of mode *out* or *inout* are interpreted as being of class *variable*. Parameter of type **FILE** has no mode.
- Examples of procedure declaration
 - Object class of parameter is implicit
procedure **Mod_256** (**X** : inout **Integer**);
 - Object class of parameter is explicit
procedure **Mod_256**(variable **X** : inout **Integer**);

Simple Example 7

→ A procedure

```
1  -- Procedure declaration
2  -- X is of class variable
3  Procedure ModTwo (X : inout Integer);
4
5  -- Procedure Specification
6  Procedure ModTwo (X : inout Integer) is
7  begin
8      case X is
9          When 0 | 1 => null;
10         When others X := X mod 2;
11     end case;
12 end ModTwo;
```

Packages

- Allows data types, subprograms, object declarations (*signal, constants, shared variables and files*), component declarations etc. to be shared by multiple design units.

package identifier is

declarations

end [package] [identifier];

- Example:

```
package logic is
    type Three_level_logic is ('0', '1', 'z');
    function invert (Input : Three_level_logic) return Three_level_logic;
end logic;
```

Package Body

- Package declarations and bodies are separately described
- Package declarations contain public and visible declarations
- Items declared inside package body is not visible outside the package body
- Package body has the same name as the corresponding package declaration

```
package body identifier is  
    declarations  
end [package body] [identifier];
```

Package Body (contd.)

→ Example of a package body for package **logic**

```
1 package body logic is
2     -- subprogram body of function invert
3     function invert (Input: Three_level_logic) return Three_level_logic is
4     begin
5         case Input is
6             when '0' => return '1';
7             when '1' => return '0';
8             when 'Z' => return 'Z';
9         end invert;
10    end logic;
```

USE Clause

- **Use** clause preceding a unit makes all the elements of a package or a particular element of a package visible to the unit
- An architecture body inherits the use clauses of its entity. If those use clauses are sufficient for the descriptions inside the architecture, then no explicit use clause is necessary for the architecture
- **Simple examples :**

```
use ieee.std_logic_1164.all;
```

Makes all items of package *std_logic_1164* in library *ieee* visible

```
use work.Logic.Three_level_logic;
```

Makes *Three_level_logic* in package *Logic* in library *work* visible



Sample codes

Appendix A

Example

```
entity state_machine is
    port(a, b, clock, reset :in BIT;
         z :out BIT);
end entity state_machine;

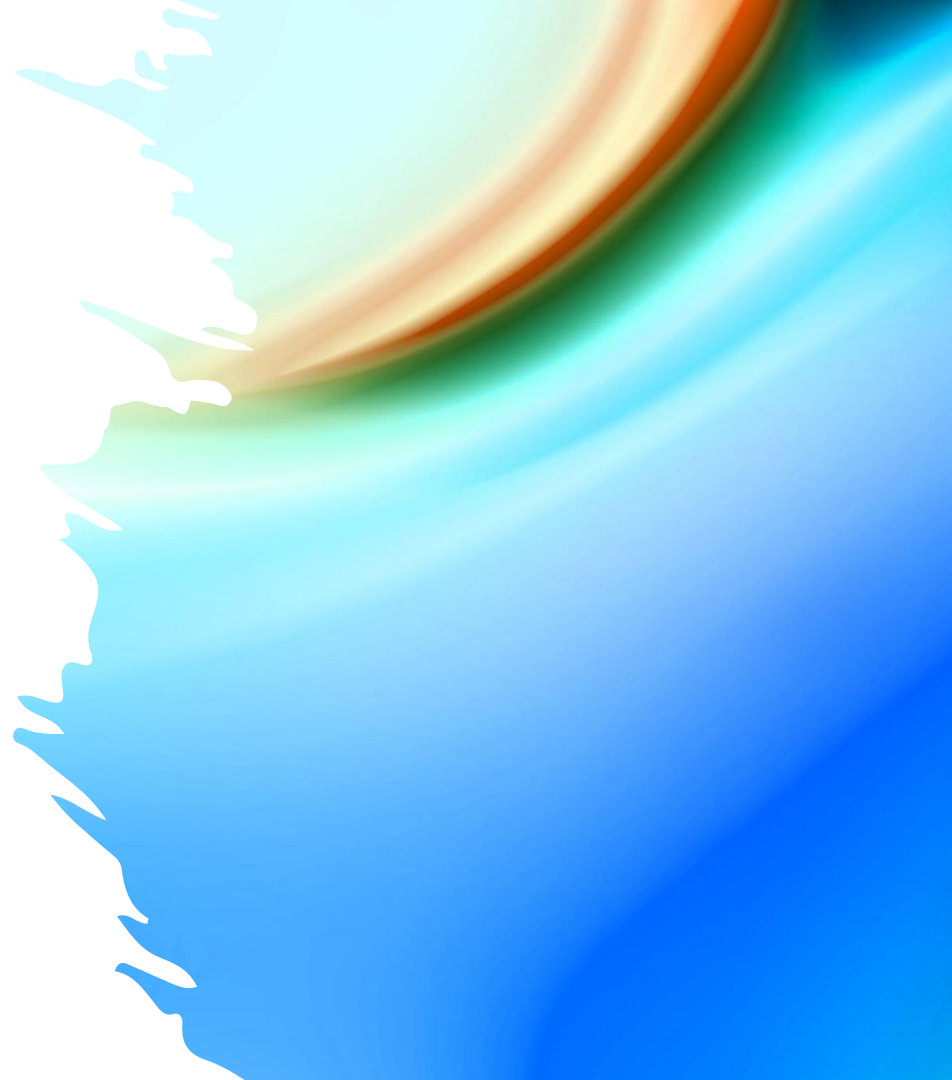
architecture two of state_machine is
    type state_type is (S0, S1, S2);
    signal state, next_state : state_type;

seq: process (clock, reset) is
begin
    if reset = '1' then
        state <= S0;
    elsif rising_edge(clock) then
        state <= next_state;
    end if;
end process seq;

com: process (state, a, b) is
begin
    Z <= '0';
    case state is
        when S0 =>
            if a = '1' and b = '1' then
                next_state <= S1;
            else
                next_state <= S0;
            end if;
        when S1 =>
            if a = '1' and b = '0' then
                next_state <= S2;
            else
                next_state <= S0;
            end if;
        when S2 =>
            if a = '0' and b = '0' then
                Z <= '1';
            end if;
            next_state <= S0;
        end case;
    end process com;
end architecture two;
```

Appendix B

List of HLS Tools



A list of High-Level Synthesis CAD tools (visited January 2024):

https://en.wikipedia.org/wiki/High-level_synthesis

Status	Compiler	Owner	License	Input	Output	Year	Domain	Test Bench	FP	FixP
In Use	Stratus HLS	Cadence Design Systems	Commercial	C/C++ SystemC	RTL	2015	All	Yes	Yes	Yes
	AUGH	TIMA Lab.	Academic	C subset	VHDL	2012	All	Yes	No	No
	eXCite	Y Explorations	Commercial	C	VHDL/Verilog	2001	All	Yes	No	Yes
	Bambu	PoliMi	Academic	C	VHDL/Verilog	2012	All	Yes	Yes	No
	Bluespec	BlueSpec Inc.	BSD-3	BSV	SystemVerilog	2007	All	No	No	No
	QCC	CacheQ Systems, Inc.	Commercial	C/C++/Fortan	Host Executable + FPGA Bit file (SystemVerilog is intermediate)	2018	All - multi-core and heterogeneous compute	Yes (C++)	Yes	Yes
	CHC	Altium	Commercial	C subset	VHDL/Verilog	2008	All	No	Yes	Yes
	CoDeveloper	Impulse Accelerated	Commercial	Impulse-C	VHDL	2003	Image Streaming	Yes	Yes	No
	HDL Coder	MathWorks	Commercial	MATLAB, Simulink, Stateflow, Simscape	VHDL / Verilog	2003	Control Systems, Signal Processing, Wireless, Radar, Communications, Image and Computer Vision	Yes	Yes	Yes
	CyberWorkBench	NEC	Commercial	C, BDL, SystemC	VHDL/Verilog	2004	All	Cycle/Formal	Yes	Yes
	Catapult	Mentor (Siemens business)	Commercial	C, C++, SystemC	VHDL/Verilog	2004	All	Yes	Yes	Yes
	DWARV	TU. Delft	Academic	C subset	VHDL	2012	All	Yes	Yes	Yes
	GAUT	U. Bretagne	Academic	C/C++	VHDL	2010	DSP	Yes	No	Yes
	Hastlayer	Lombiq Technologies	BSD-3	C#/C++/F#... (.NET)	VHDL	2015	.NET	Yes	Yes	Yes
	Instant SoC	FPGA Cores	Commercial	C/C++	VHDL/Verilog	2019	All	Yes	No	No
	Intel High Level Synthesis Compiler	Intel FPGA (Formerly Altera)	Commercial	C/C++	Verilog	2017	All	Yes	Yes	Yes

A list of High-Level Synthesis CAD tools (visited January 2024):

https://en.wikipedia.org/wiki/High-level_synthesis

	LegUp HLS	LegUp Computing	Commercial	C/C++	Verilog	2015	All	Yes	Yes	Yes
	LegUp	U. Toronto	Academic	C	Verilog	2010	All	Yes	Yes	No
	MaxCompiler	Maxeler	Commercial	MaxJ	RTL	2010	DataFlow	No	Yes	No
	ROCCC	Jacquard Comp.	Commercial	C subset	VHDL	2010	Streaming	No	Yes	No
	Symphony C	Synopsys	Commercial	C/C++	VHDL/Verilog/SystemC	2010	All	Yes	No	Yes
	VivadoHLS (formerly AutoPilot from AutoESL ^[14])	Xilinx	Commercial	C/C++/SystemC	VHDL/Verilog/SystemC	2013	All	Yes	Yes	Yes
	Kiwi	U. Cambridge	Academic	C#	Verilog	2008	.NET	No	Yes	Yes
	CHIMPS	U. Washington	Academic	C	VHDL	2008	All	No	No	No
	gcc2verilog	U. Korea	Academic	C	Verilog	2011	All	No	No	No
	HercuLeS	Ajax Compilers	Commercial	C/NAC	VHDL	2012	All	Yes	Yes	Yes
	Shang	U. Illinois	Academic	C	Verilog	2013	All	Yes	?	?
Abandoned	Trident	Los Alamos NL	Academic	C subset	VHDL	2007	Scientific	No	Yes	No
	AccelDSP	Xilinx	Commercial	MATLAB	VHDL/Verilog	2006	DSP	Yes	Yes	Yes
	C2H	Altera	Commercial	C	VHDL/Verilog	2006	All	No	No	No
	CtoVerilog	U. Haifa	Academic	C	Verilog	2008	All	No	No	No
	DEFACTO	U. South Calif.	Academic	C	RTL	1999	DSE	No	No	No
	Garp	U. Berkeley	Academic	C subset	bitstream	2000	Loop	No	No	No
	MATCH	U. Northwest	Academic	MATLAB	VHDL	2000	Image	No	No	No
	Napa-C	Sarnoff Corp.	Academic	C subset	VHDL/Verilog	1998	Loop	No	No	No
	PipeRench	U. Carnegie M.	Academic	DIL	bistream	2000	Stream	No	No	No
	SA-C	U. Colorado	Academic	SA-C	VHDL	2003	Image	No	No	No
	SeaCucumber	U. Brigham Y.	Academic	Java	EDIF	2002	All	No	Yes	Yes
	SPARK	U. Cal. Irvine	Academic	C	VHDL	2003	Control	No	No	No